Step-by-Step Build Guide — AI Mystery Room

Project Submission Link: https://bit.ly/ai-mystery-room-sub-form

Project Deadline: ***2nd November, 11:59PM***

1. Project Overview

AI Mystery Room is an AI-powered web application where users create and solve mysteries generated using Gemini. Players investigate suspects, discover clues, request hints, and identify the culprit.

The objective is to plan the project using a detailed spec sheet, build the application, deploy it, and demonstrate a complete mystery-solving experience on the live website.

2. Phase 1: Create a Detailed Spec Sheet

Before writing any code, create a `SPEC.md` file. This document serves as the blueprint for the entire project and guides AI coding tools such as Codex through implementation.

A. Project Requirements

- Project overview and problem statement.
- Target users and expected user experience.
- Core features and optional enhancements.
- Technology stack: React, Vite, Tailwind CSS, Node.js, Express, MongoDB, Mongoose, and Gemini API.
- Project limitations and final deliverables.

B. User Flow

Document the complete journey:

1. User opens the website.
2. User enters a mystery premise and selects a difficulty level.
3. Gemini generates the mystery, suspects, clues, and hints.
4. User investigates and discovers clues.
5. User requests hints if needed.

6. User accuses a suspect.
7. The application reveals the verdict, culprit, explanation, and score.
8. User can start another mystery.

C. UI and Page Specifications

Define the purpose and essential elements of each page:

- **Home:** Project introduction and start button.
- **Create Mystery:** Premise input and difficulty selector.
- **Investigation Room:** Story, suspect cards, clues, hints, and progress.
- **Accusation:** Suspect selection and submission.
- **Results:** Verdict, score, culprit, and explanation.

Specify loading states, error messages, navigation, and responsive behavior. Students may use Stitch AI to generate an initial UI design, then document how each screen maps to the project requirements.

D. Backend and Database Design

Document the API endpoints, request and response formats, validation rules, and error handling.

Suggested endpoints:

- `POST /api/cases` — Generate a mystery.
- `GET /api/cases/:id` — Retrieve an active mystery.
- `POST /api/cases/:id/clues/:clueId/discover` — Discover a clue.
- `POST /api/cases/:id/hints` — Request a hint.
- `POST /api/cases/:id/accuse` — Submit an accusation.
- `GET /api/cases/:id/result` — Retrieve the completed result.

Define a database model containing the mystery title, story, difficulty, suspects, clues, hints, player progress, accusation, and score.

Security requirement: Keep the culprit and solution on the backend until the player submits an accusation.

E. AI Behavior and Scoring Rules

Specify the expected Gemini output format and validate generated responses before saving them.

Difficulty	Suspects	Clues	Hints
Easy	3	5	2
Medium	4	7	3
Hard	5	9	3

Define scoring rules in the spec sheet:

- Correct accusation: Easy 100, Medium 150, Hard 200 points.
- Each discovered clue: +5 points.
- Each hint used: −10 points.
- Minimum score: 0.

Require a logically consistent solution supported by the clues. Scoring and validation must happen on the backend.

F. Acceptance Criteria

Define clear conditions that determine whether the project is complete:

- Users can generate mysteries at all three difficulty levels.
- Players can investigate, request hints, and submit accusations.
- The solution is not revealed prematurely.
- Scores are calculated and validated by the backend.
- Results display the correct culprit and explanation.
- The application works on the deployed website.

Spec sheet deliverable: A complete `SPEC.md` that another developer or AI coding assistant can follow without major clarification.

3. Phase 2: Set Up the Project

Create separate frontend and backend applications.

Frontend setup:

```
npm create vite@latest client -- --template react
cd client
npm install
npm install react-router-dom axios
```

Backend setup:

mkdir server
cd server
npm init -y
npm install express cors dotenv mongoose @google/genai zod
npm install --save-dev nodemon

Create a backend `.env` file:

PORT=5000
MONGODB_URI=your_mongodb_connection_string
GEMINI_API_KEY=your_gemini_api_key
CLIENT_URL=http://localhost:5173

Never expose API keys in frontend code or commit secrets to GitHub.

4. Phase 3: Build the Application

Follow the `SPEC.md` step by step:

1. Set up Express and connect MongoDB.
2. Create the database model and API endpoints.
3. Integrate Gemini to generate structured mystery data.
4. Validate AI output and keep the solution private.
5. Build the React pages and connect them to the APIs.
6. Implement clue discovery, hints, accusations, and scoring.
7. Display the final result and allow users to start another mystery.
8. Test every feature against the acceptance criteria.

Students can use Codex or another AI coding assistant, but they should provide it with the spec sheet first. Build and test the application incrementally instead of generating everything in one step.

5. Phase 4: Test and Deploy

Verify that:

- All difficulty levels work.
- Invalid AI responses are handled safely.
- Clues, hints, accusations, and scoring work correctly.
- The solution is not exposed before the accusation.
- The application handles API failures gracefully.
- The complete game works after deployment.

Deploy the backend to a suitable hosting provider, configure MongoDB Atlas, and deploy the frontend to Vercel. Configure environment variables and test the complete workflow using the production website.

6. Final Submission

Complete the following steps:

1. Create and finalize `SPEC.md`.
2. Build the application according to the spec sheet.
3. Test the complete mystery-solving flow.
4. Deploy the application.
5. Create and solve a mystery on the deployed website.
6. Submit:
    - **GitHub Repository:** Complete source code.
    - **Live Website URL:** Deployed application.

Final goal: Students should demonstrate that they can plan a project using a clear spec sheet, build an AI-powered application systematically, validate its behavior, and deploy a working experience.

Project Submission Link: https://bit.ly/ai-mystery-room-sub-form

Project Deadline: 2nd November, 11:59PM

Pinned Chats:
https://online-clipboard.online/online-clipboard/
468442
https://docs.google.com/document/d/1sDP9-tn82yphSZpMXBLZKpukII9QIC0ncg-SC7XJptw/edit?usp=sharing

https://bit.ly/ai-mystery-room-sub-form